PROGRAM KONVOLUSI MENGGUNAKAN LINEAR FILTER 5 X 5

Dosen Pengampu:
Afdhal Dinilhak, M. Kom.

Muhammad Luthfi Kautsar Rizata
2311532020

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

SEMESTER GENAP 2025/2026

A. Materi Pengantar

1. Pengertian Citra Digital

Citra dapat didefinisikan sebagai suatu representasi, kemiripan, atau imitasi dari suatu objek nyata. Secara harfiah, citra adalah fungsi intensitas cahaya dua dimensi yang dinotasikan dengan f(x, y), di mana x dan y merupakan koordinat spasial, dan nilai f pada titik (x, y) menyatakan intensitas atau tingkat kecerahan citra pada titik tersebut. Citra bisa juga didefinisikan sebagai representasi dari citra kontinu dalam bentuk diskrit yang dapat diolah oleh komputer. Suatu citra digital didefinisikan sebagai suatu array atau matriks dua dimensi yang tersusun atas elemen-elemen diskrit yang disebut piksel . Setiap piksel memiliki dua atribut utama, yaitu posisi (koordinat) dan nilai intensitas (warna atau keabuan).

Berdasarkan jenis nilai intensitasnya, citra digital dapat diklasifikasikan menjadi beberapa jenis, antara lain:

1. **Citra Biner:** Setiap piksel hanya bernilai 0 atau 1, merepresentasikan warna hitam dan putih.
2. **Citra Grayscale:** Setiap piksel mewakili intensitas cahaya, biasanya dalam rentang 0 (hitam) hingga 255 (putih).
3. **Citra Warna (RGB):** Setiap piksel terdiri dari tiga komponen warna dasar: *Red* (merah), *Green* (hijau), dan *Blue* (biru). Masing-masing komponen memiliki rentang nilai 0-255.

2. Konvolusi Citra

Konvolusi merupakan operasi matematis fundamental dalam pengolahan citra yang digunakan untuk menerapkan berbagai macam efek, seperti smoothing, sharpening, dan deteksi tepi. Konvolusi bekerja dengan mengkombinasikan dua buah fungsi untuk menghasilkan fungsi ketiga, yang mencerminkan bagaimana bentuk satu fungsi dimodifikasi oleh fungsi lainnya.

Dalam ranah diskrit pengolahan citra, konvolusi didefinisikan sebagai operasi antara suatu citra masukan I dengan suatu matriks kecil yang disebut kernel atau filter (dinotasikan K), yang menghasilkan citra keluaran I'. Proses ini dilakukan dengan menggeser kernel di atas seluruh area citra. Untuk setiap posisi piksel, dilakukan perkalian antara elemen-elemen kernel dengan nilai piksel yang tertutup oleh kernel tersebut, kemudian seluruh hasil perkalian dijumlahkan untuk mendapatkan nilai piksel baru pada citra keluaran.

Secara matematis, operasi konvolusi 2D untuk citra I dan kernel K (dengan asumsi kernel berukuran ganjil, misal m x n) dinyatakan sebagai:

$$(I * K)(x, y) = \Sigma \Sigma I(x + i, y + j) \cdot K(i, j)$$

di mana:

- $(I * K)(x, y)$ adalah nilai piksel hasil konvolusi pada koordinat (x, y).
- $I(x + i, y + j)$ adalah nilai piksel tetangga dari citra asli.
- $K(i, j)$ adalah nilai bobot pada kernel di posisi (i, j).

3. Kernel

Dalam konteks pengolahan citra, kernel adalah sebuah matriks kecil berukuran ganjil (misalnya 3x3, 5x5, 7x7) yang berisi nilai-nilai numerik yang berfungsi sebagai bobot. Kernel adalah inti dari operasi konvolusi dan menentukan efek yang akan dihasilkan pada citra.

Ukuran kernel yang umum digunakan adalah 3x3 karena kesederhanaan dan efisiensi komputasinya. Penggunaan ukuran ganjil bertujuan agar kernel memiliki titik pusat (anchor) yang jelas, sehingga memudahkan proses pergeseran dan perhitungan.
Berdasarkan fungsinya, kernel dapat dikategorikan menjadi beberapa jenis, di antaranya:

- **Low-pass filter:** Bertujuan untuk menghaluskan citra dan mengurangi *noise* (derau). Contoh: Mean filter, Gaussian filter.
- **High-pass filter:** Bertujuan untuk menajamkan citra dan mendeteksi tepi. Contoh: Sobel filter, Laplacian filter.

B. Penjelasan Kode

1. Library NumPy digunakan untuk memproses data numerik dalam bentuk array dan matriks karena citra digital direpresentasikan sebagai kumpulan nilai piksel berbentuk matriks. Library PIL (Python Imaging Library) digunakan untuk membuka, membaca, dan menyimpan file gambar dengan berbagai format. Sementara itu, Matplotlib dimanfaatkan untuk menampilkan citra baik sebelum maupun sesudah proses konvolusi sehingga hasil pemrosesan dapat divisualisasikan secara langsung.

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

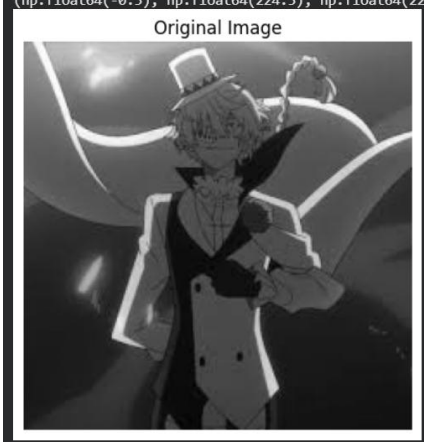
2. Fungsi Image.open("images.jpg") dari library PIL berfungsi untuk membuka file gambar yang akan diolah. Selanjutnya, method .convert("L") digunakan untuk mengubah citra berwarna (RGB) menjadi citra grayscale. Konversi ini dilakukan karena proses konvolusi lebih sederhana diterapkan pada citra satu kanal intensitas dibandingkan citra tiga kanal warna. Setelah citra dikonversi, perintah np.array(img) digunakan untuk mengubah objek citra menjadi array NumPy

```
img = Image.open("images.jpg").convert("L")
img_array = np.array(img)
```

3. Fungsi plt.imshow(img_array, cmap='gray') berfungsi untuk menampilkan data citra yang telah disimpan dalam bentuk array NumPy. Parameter cmap='gray' digunakan agar citra grayscale ditampilkan dengan peta warna abu-abu sehingga perbedaan intensitas piksel terlihat jelas. Selanjutnya, perintah plt.title("Original Image") digunakan untuk menambahkan judul pada gambar agar pengguna dapat membedakan antara citra asli dan citra hasil pemrosesan. Perintah plt.axis('off') berfungsi untuk menyembunyikan sumbu koordinat sehingga tampilan citra menjadi lebih rapi dan fokus pada gambar. Output berupa nilai koordinat seperti (np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5)) sebagai informasi batas sumbu yang dikembalikan oleh Matplotlib setelah pengaturan axis dilakukan. Nilai tersebut menunjukkan rentang koordinat area tampilan citra.

```
plt.imshow(img_array, cmap='gray')
plt.title("Original Image")
plt.axis('off')

(np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5))
```



4. Perintah np.ones((5,5)) menghasilkan matriks berukuran 5×5 yang seluruh elemennya bernilai 1. Matriks tersebut kemudian dibagi dengan 25 sehingga setiap elemennya bernilai 0,04. Pembagian ini bertujuan untuk meratakan nilai intensitas piksel di sekitar area konvolusi agar hasil akhir menjadi lebih halus (blur). Perintah print(kernel) menampilkan isi kernel sehingga dapat dipastikan bahwa seluruh elemen bernilai sama, yaitu 0,04. Nilai tersebut diperoleh dari hasil pembagian 1/25 karena jumlah elemen pada kernel 5×5 adalah 25. Selanjutnya, perintah np.pad(img_array, pad_width=2, mode='constant', constant_values=0) digunakan untuk menambahkan padding di sekeliling citra. Padding selebar 2 piksel diperlukan karena ukuran kernel adalah 5×5, sehingga pusat kernel dapat menjangkau piksel tepi citra saat proses konvolusi. Mode 'constant' dengan nilai 0 berarti area tambahan di luar citra asli akan diisi dengan piksel berwarna hitam.

```
kernel = np.ones((5,5)) / 25
print(kernel)

padded_img = np.pad(img_array, pad_width=2, mode='constant', constant_values=0)

[[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]
```

5. Perintah np.zeros_like(img_array) digunakan untuk membuat matriks keluaran dengan ukuran yang sama seperti citra asli, di mana seluruh elemennya diinisialisasi dengan nilai nol. Matriks ini akan menyimpan hasil akhir proses konvolusi. Proses konvolusi dilakukan menggunakan dua buah perulangan bersarang. Perulangan pertama mengiterasi baris citra, sedangkan perulangan kedua mengiterasi kolom citra. Dengan cara ini, setiap piksel pada citra akan diproses satu per satu. Di dalam perulangan, perintah region = padded_img[i:i+5, j:j+5] digunakan untuk mengambil bagian citra berukuran 5×5 dari citra yang telah diberi padding. Selanjutnya, proses konvolusi dilakukan dengan perintah region * kernel yang mengalikan setiap elemen piksel pada region dengan elemen kernel yang bersesuaian. Hasil perkalian tersebut kemudian dijumlahkan menggunakan fungsi np.sum() untuk menghasilkan satu nilai piksel baru. Nilai inilah yang disimpan ke dalam matriks keluaran pada posisi yang sama melalui perintah output[i, j] = ...

```
output = np.zeros_like(img_array)

for i in range(img_array.shape[0]):
    for j in range(img_array.shape[1]):
        region = padded_img[i:i+5, j:j+5]
        output[i,j] = np.sum(region * kernel)
```

6. plt.imshow(output, cmap='gray') berfungsi untuk menampilkan matriks hasil konvolusi sebagai citra grayscale. Parameter cmap='gray' memastikan bahwa nilai intensitas piksel divisualisasikan dalam gradasi abu-abu sehingga efek perataan citra dapat terlihat dengan jelas. Perintah plt.title("Filtered Image (5x5 Mean Filter)") digunakan untuk memberikan judul pada gambar hasil pemrosesan agar mudah dibedakan dari citra asli. Sementara itu, perintah plt.axis('off') bertujuan untuk menyembunyikan sumbu koordinat sehingga tampilan citra menjadi lebih rapi dan fokus pada hasil filter. Nilai keluaran berupa (np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5)) bukan merupakan kesalahan, melainkan informasi batas koordinat tampilan yang dikembalikan oleh Matplotlib.

```
plt.imshow(output, cmap='gray')
plt.title("Filtered Image (5x5 Mean Filter)")
plt.axis('off')

(np.float64(-0.5), np.float64(224.5), np.float64(224.5), np.float64(-0.5))
```

7. Image.fromarray(output) berfungsi untuk mengubah array NumPy yang berisi hasil pemrosesan citra menjadi objek gambar yang dapat dikenali oleh library PIL. Proses ini diperlukan karena PIL hanya dapat menyimpan data dalam bentuk objek citra, bukan dalam bentuk array numerik. Selanjutnya, perintah result_img.save("hasil_konvolusi.jpg") digunakan untuk menyimpan objek citra tersebut ke dalam file dengan format JPG. File yang dihasilkan merupakan citra baru yang telah melalui proses konvolusi menggunakan kernel rataan 5×5, sehingga dapat digunakan untuk analisis lebih lanjut

```
result_img = Image.fromarray(output)
result_img.save("hasil_konvolusi.jpg")
```

C. Hasil Analisis

1. Perbandingan Citra Sebelum dan Sesudah Filter

Berikut adalah hasil analisis perbandingan citra sebelum dan sesudah diterapkan konvolusi menggunakan *mean filter* berukuran 5×5:

Aspek yang Diamati	Citra Asli (Sebelum Filter)	Citra Hasil Filter (Sesudah Filter)
Tampilan Visual	Tampak tajam dan jelas, detail objek terlihat dengan baik, termasuk tekstur dan tepi-tepi objek.	Tampak lebih halus dan lembut, namun terlihat kabur (*blur*). Detail halus seperti tekstur permukaan berkurang.
Ketajaman	Tinggi. Perbedaan intensitas antar piksel yang berdekatan cukup signifikan, terutama pada area tepi objek.	Rendah. Transisi antar piksel menjadi lebih landai karena nilai piksel telah dirata-rata dengan piksel tetangganya.
Tingkat Noise	*Noise* masih terlihat sebagai titik-titik kasar yang mengganggu kehalusan citra.	*Noise* berkurang secara signifikan. Derau halus akan tersamar karena proses perataan nilai.
Detail Tekstur	Detail tekstur seperti serat kayu, pori-pori, atau pola permukaan objek masih terlihat jelas.	Detail tekstur berkurang atau bahkan hilang karena proses perataan cenderung menghomogenkan area dengan variasi intensitas tinggi.

Tepi Objek	Tepi objek tegas dan mudah dikenali karena kontras yang tinggi antara objek dan latar belakang.	Tepi objek terlihat kurang tegas dan cenderung melebar karena pengaruh piksel di sekitarnya.

2. Efek Blur dari Mean Filter

Mean filter termasuk dalam kategori low-pass filter. Sesuai dengan namanya, filter ini melewatkan komponen frekuensi rendah (area homogen) dan meredam komponen frekuensi tinggi. Efek *blur* yang dihasilkan disebabkan oleh mekanisme kerja filter sebagai berikut:

a. Prinsip Perataan Nilai Piksel

Mean filter bekerja dengan mengganti nilai suatu piksel dengan nilai rata-rata dari piksel tersebut dan piksel-piksel tetangganya yang tercakup dalam kernel. Kernel 5×5 yang digunakan memiliki 25 elemen dengan bobot yang sama, yaitu 0,04. Seluruh elemen kernel ini jika dijumlahkan hasilnya adalah 1, sehingga kecerahan citra secara keseluruhan tidak berubah.

$$Kernel = \frac{1}{25} \times \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.04 & 0.04 & 0.04 & 0.04 & 0.04 \\ 0.04 & 0.04 & 0.04 & 0.04 & 0.04 \\ 0.04 & 0.04 & 0.04 & 0.04 & 0.04 \\ 0.04 & 0.04 & 0.04 & 0.04 & 0.04 \\ 0.04 & 0.04 & 0.04 & 0.04 & 0.04 \end{bmatrix}$$

b. Penghalusan Variasi Intensitas

Ketika kernel melewati area dengan variasi intensitas tinggi (misalnya tepi objek atau tekstur), proses perhitungan rata-rata akan "mencampur" nilai piksel terang dengan piksel gelap di sekitarnya. Akibatnya, nilai piksel baru berada di antara nilai ekstrem tersebut. Proses ini secara efektif menghaluskan perubahan intensitas yang tiba-tiba.

c. Pengaruh Ukuran Kernel terhadap Tingkat Blur

Ukuran kernel sangat memengaruhi seberapa kuat efek *blur* yang dihasilkan:

- Kernel 3×3: efek *blur* ringan, detail masih cukup terjaga.
- Kernel 5×5: efek *blur* sedang, detail mulai berkurang.
- Kernel 7×7 atau lebih besar: efek *blur* sangat kuat, citra tampak sangat kabur.

Pada program ini digunakan kernel 5×5, sehingga menghasilkan tingkat *blur* yang cukup signifikan namun objek utama dalam citra masih dapat dikenali.

Link github: https://github.com/sonicmortis/2311532020-Luthfi-Kautsar-Rizata-ImageProcessing